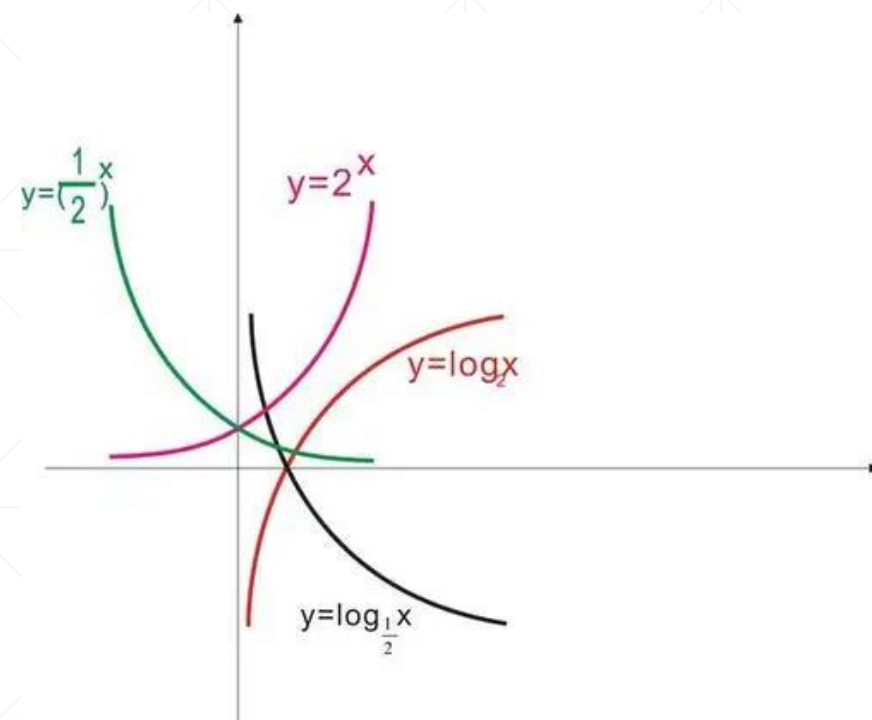


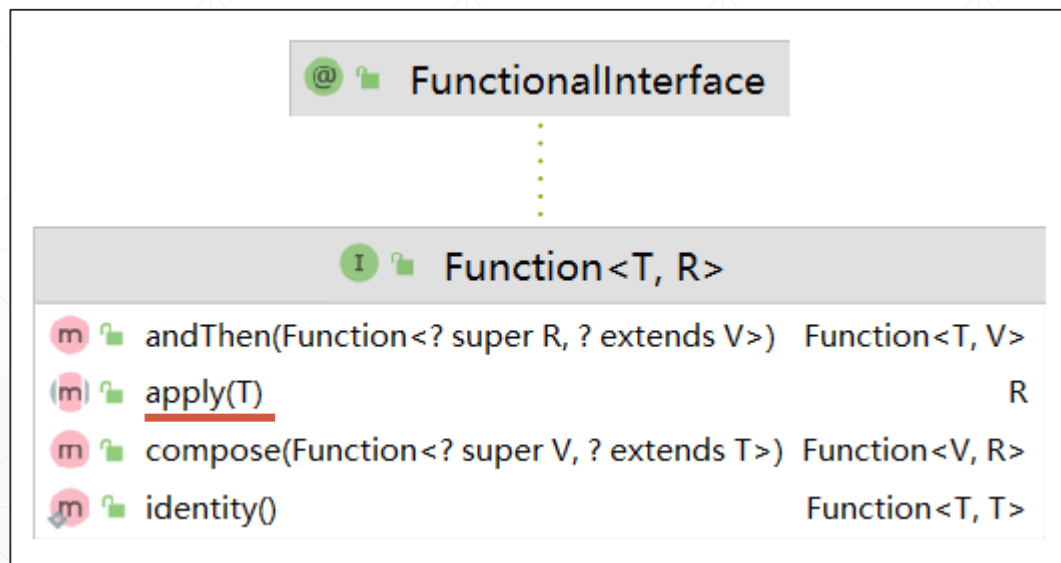


Function及相关接口

北京理工大学计算机学院
金旭亮

Function<T, R>接口

此接口定义了一个apply方法，它接收一个T类型的对象，返回一个R类型的对象：

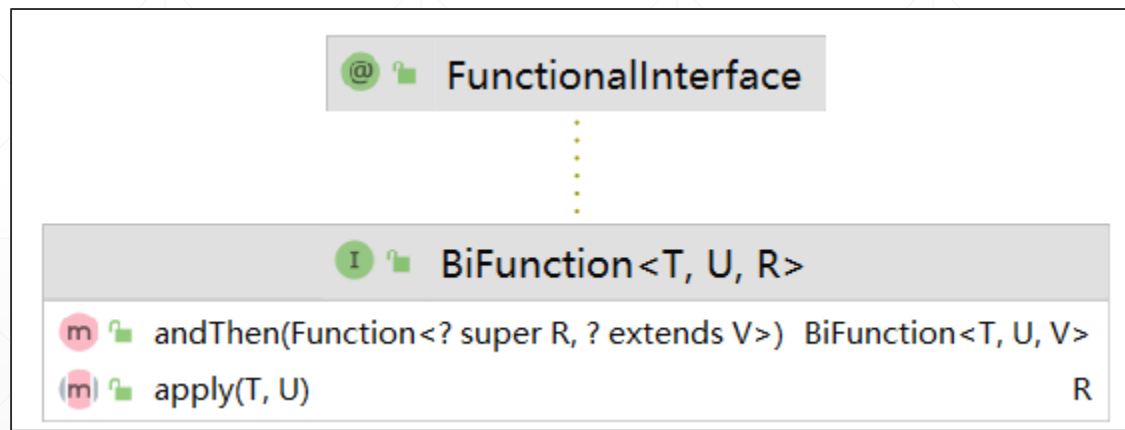


Function<T,R>实例

```
private static void useFunctionInterface() {  
    //将一个英文句子按空格分割, 然后统计得到的单词的个数  
    Function<String, Integer> wordCount =  
        s -> s.split(regex: " ").length;  
    String str = "this is a test.";  
    System.out.println(wordCount.apply(str)); //4  
}
```

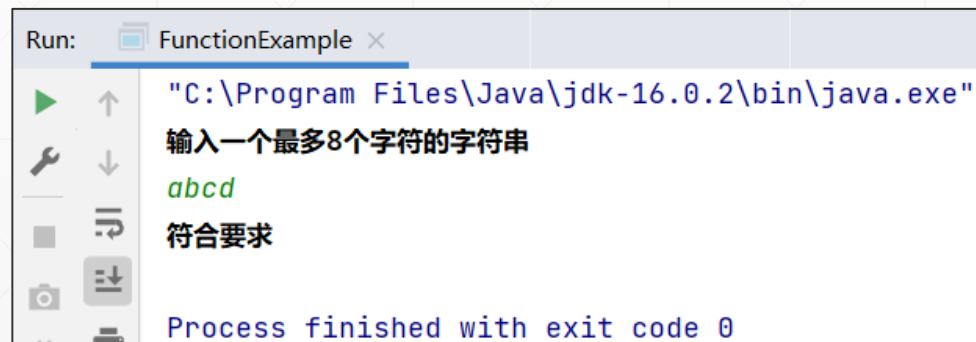
BiFunction接口

作为Function接口的特例，有一个BiFunction<T, U, R>接口，它所定义的apply方法接收两个参数：T和U类型的，然后返回一个R类型的对象。



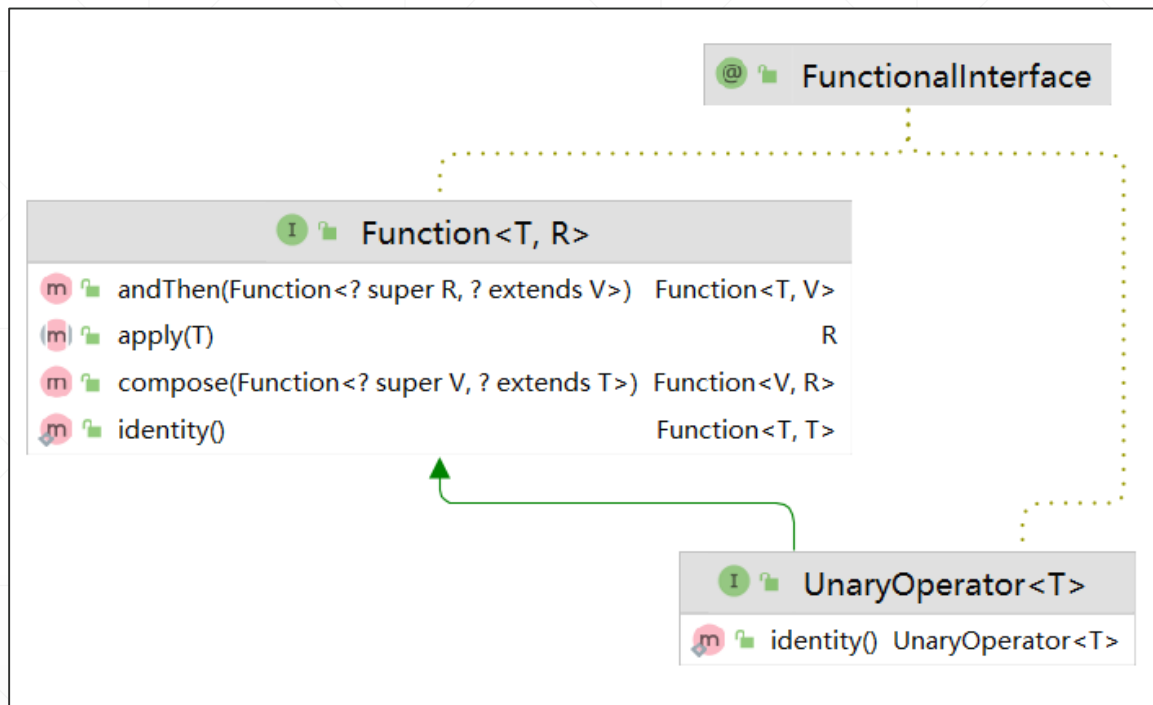
BiFunction<T, U, R>接口示例

```
private static void useBiFunctionInterface() {  
    //检查某字符串是否长度超限  
    BiFunction<String, Integer, Boolean> exceedsMaxLength =  
        (s, maxLength) -> {  
            int actualLength = s.length();  
            return actualLength > maxLength;  
        };  
  
    Scanner scanner = new Scanner(System.in);  
    System.out.println("输入一个最多8个字符的字符串");  
    String userInput = scanner.nextLine();  
    boolean result = exceedsMaxLength.apply(userInput, u: 8);  
    if (!result) {  
        System.out.println("符合要求");  
    } else {  
        System.out.println("无效输入");  
    }  
}
```



```
Run: FunctionExample x  
"C:\Program Files\Java\jdk-16.0.2\bin\java.exe"  
输入一个最多8个字符的字符串  
abcd  
符合要求  
Process finished with exit code 0
```

UnaryOperator接口

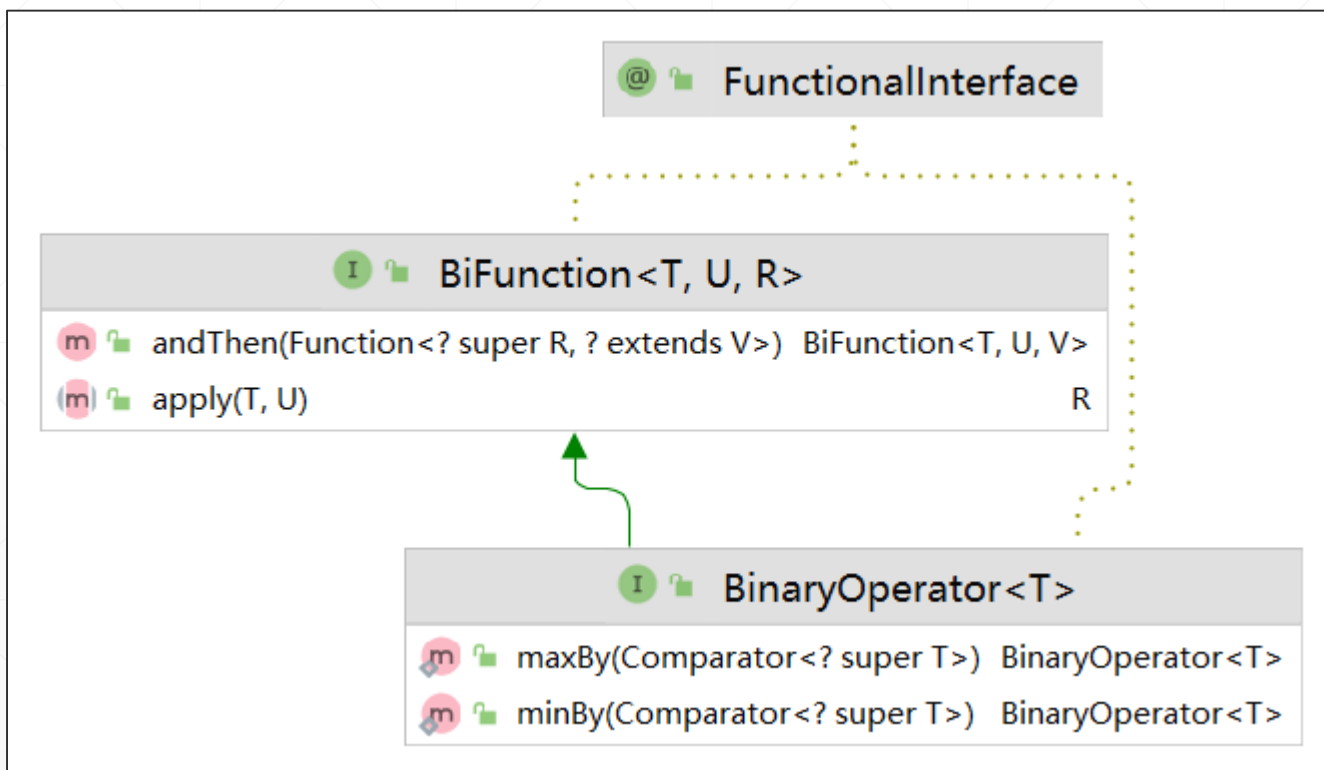


UnaryOperator接口派生自Function接口，它的返回值类型与参数类型一致，其实就是Function<T,T>的简写。

```
private static void useUnaryOperator() {  
    //接收一个参数，返回同类型的单个结果  
    UnaryOperator<Integer> autoIncrease = num -> ++num;  
    System.out.println(autoIncrease.apply(100));  
}
```

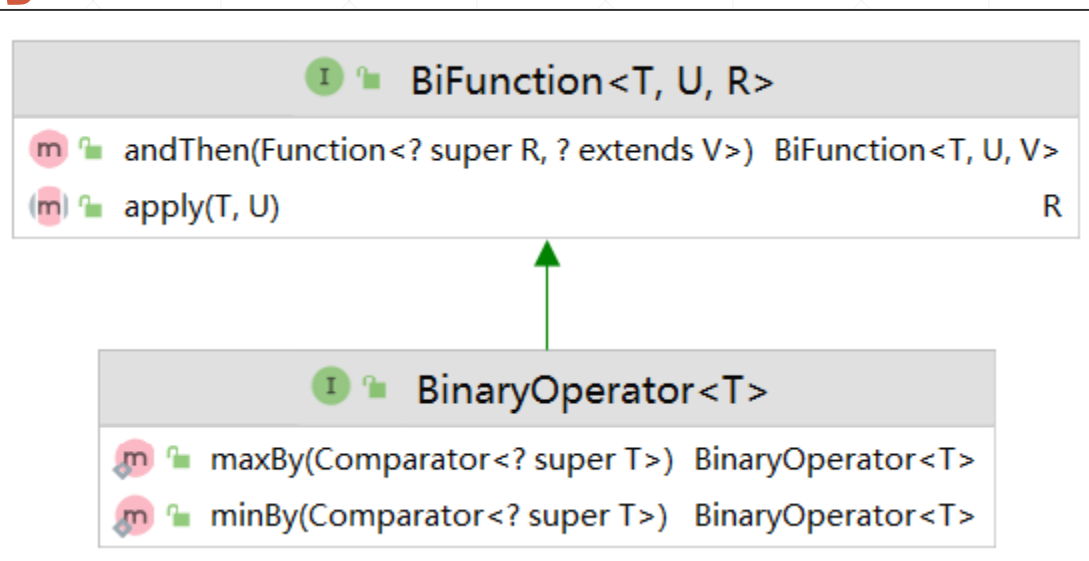
BinaryOperator接口

`BinaryOperator<T>`，等价于`BiFunction<T,T,T>`，即所有参数类型与函数返回值类型均一致。



BinaryOperator接口示例

```
private static void useBinaryOperator() {  
    //接收两个参数，返回单个结果，三个都是同类型的  
    BinaryOperator<Integer> add = (x, y) -> x + y;  
    var sum : Integer = add.apply( t: 100, u: 200);  
    System.out.println(sum); //300  
    //BinaryOperator定义了两个静态方法，可以用于求两个数值的最大值或最小值  
    Comparator<Integer> comparator = Integer::compareTo;  
    BinaryOperator<Integer> maxBy = BinaryOperator.maxBy(comparator);  
    System.out.println(maxBy.apply( t: 100, u: 200)); //200  
}
```

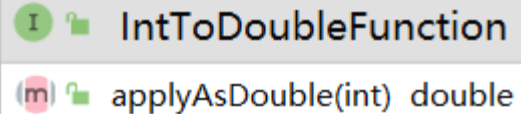


`minBy`和`maxBy`都是`BinaryOperator`接口所定义的静态方法。

IntToDoubleFunction接口

➔ java.util.function中还定义了一批与int等原始数据类型相关的接口，使用这些接口，可以避免装箱和拆箱操作造成的性能损失。

➔ IntToDoubleFunction就是其中的一个，它接收一个int类型的数值，处理之后，返回一个double类型的数值：



IntToDoubleFunction
applyAsDouble(int) double

```
public class IntToDoubleDemo {  
    public static void main(String[] args) {  
        IntToDoubleFunction func = num -> num * 0.5;  
        double result = func.applyAsDouble( value: 100);  
        System.out.println(result);  
    }  
}
```

举一反三



接口	单一抽象方法
IntFunction	<code>R apply(int value)</code>
DoubleFunction	<code>R apply(double value)</code>
LongFunction	<code>R apply(long value)</code>
ToIntFunction	<code>int applyAsInt(T value)</code>
ToDoubleFunction	<code>double applyAsDouble(T value)</code>
ToLongFunction	<code>long applyAsLong(T value)</code>
DoubleToIntFunction	<code>int applyAsInt(double value)</code>
DoubleToLongFunction	<code>long applyAsLong(double value)</code>
IntToDoubleFunction	<code>double applyAsDouble(int value)</code>
IntToLongFunction	<code>long applyAsLong(int value)</code>
LongToDoubleFunction	<code>double applyAsDouble(long value)</code>
LongToIntFunction	<code>int applyAsInt(long value)</code>
BiFunction	<code>R apply(T t, U u)</code>

接口	单一抽象方法
ToIntBiFunction	<code>int applyAsInt(T t, U u)</code>
ToDoubleBiFunction	<code>double applyAsDouble(T t, U u)</code>
ToLongBiFunction	<code>long applyAsLong(T t, U u)</code>

JDK中定义了许多与Function相关的接口，请自己编写示例，掌握这些接口的用法。